

Cos214 - Prac4: TaskForge

Lucian van der Merwe u25054262,
Conrad Botha u25017749,
Nontokozo Madela u25677404

Task1:

Domain: Construction project management

A construction company runs Projects. Each Project breaks down into Phases (e.g. Foundation, Structure, Finishing), each Phase breaks down into Zones (e.g. Zone A, Zone B — physical areas of the site), and each Zone contains individual Tasks (e.g. "Excavate footing", "Pour concrete", "Install wiring"). Phases and Zones are groups — they can contain further groups or individual tasks. Tasks are the leaves — the actual units of work someone performs.

Project->Phase->Zone->Task

This system must solve a few problems including: Track a tasks real lifecycle and stop nonsensical transitions, let site manager bolt on extra responsibilities to any unit of work at run time etc

GoF Pattern

Pattern	Where it lives	Why
Composite	WorkComponent / WorkGroup / Task	Project->Phase->Zone->Task is a genuine recursive part-whole tree, cost/duration must be computed uniformly over leaves and groups.
Iterator	WorkIterator / DepthFirstIterator / PendingInspectionIterator	Two independent, differently-purposed traversals over the same tree, without exposing WorkGroup's internal vector.
State	TaskState / PlannedState / InProgressState / BlockedState / CompletedState	A Task's lifecycle governs which operations are legal, behaviour changes per state.
Decorator	WorkComponentDecorator / SafetyInspectionDecorator / PermitComplianceDecorator / SubcontractorDecorator	Optional, stackable responsibilities added to any WorkComponent (a single task or a whole phase) at runtime, without touching the class hierarchy.

Rationale:

Composite ownership: WorkGroup owns its children (raw pointers, deleted in ~WorkGroup(), which recurses because every child is polymorphically deleted through the WorkComponent* virtual destructor). This is a strict tree — no child has two parents — so single ownership is safe and simple.

Decorator ownership: A WorkComponentDecorator owns the WorkComponent* it wraps (composition), and deletes it in its destructor. When you stack decorators (new SafetyInspectionDecorator(new PermitComplianceDecorator(myTask))), deleting the outermost decorator cascades down the chain automatically. Whatever a decorator wraps whether it's a leaf Task or a whole WorkGroup is deleted exactly once, by whichever object currently owns it (either its original parent WorkGroup, or the decorator that has since replaced it in that slot).

State ownership: TaskState concrete classes are stateless (no per-task data), so they're implemented as function-static singletons (PlannedState::instance()), avoiding needless new/delete on every transition.

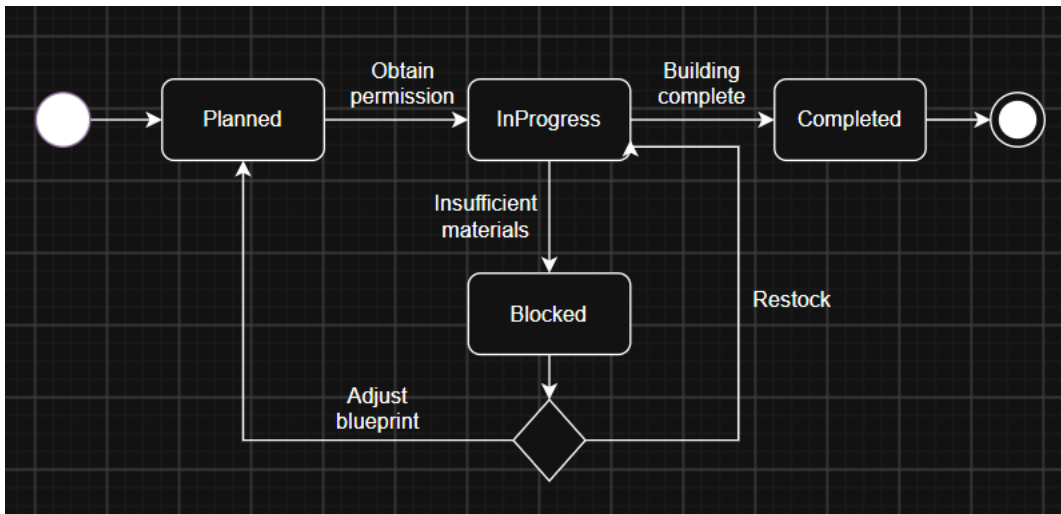
Task holds a non-owning pointer to whichever singleton is current.

Iterator ownership: the iterator owns its own snapshot vector (a flat, pre-order copy of WorkComponent* pointers, not copies of the objects) — it does not own the components themselves.

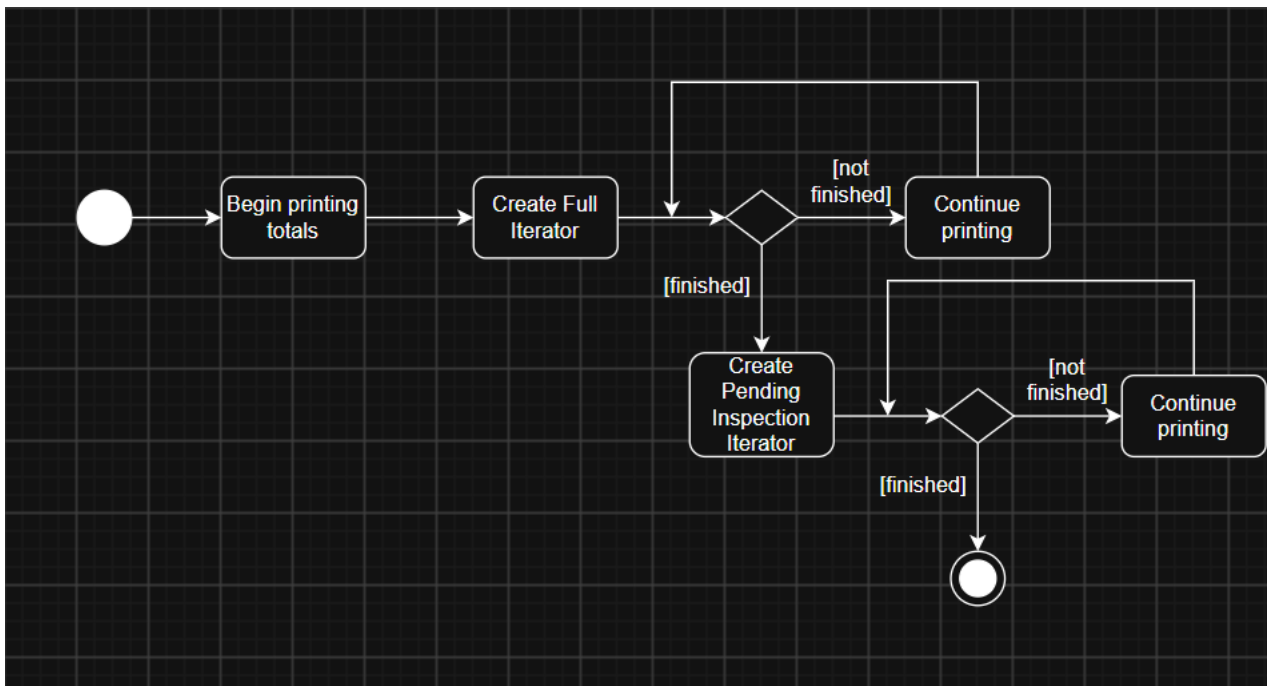
Every base class (WorkComponent, TaskState, WorkIterator) has a virtual destructor.

Task 4

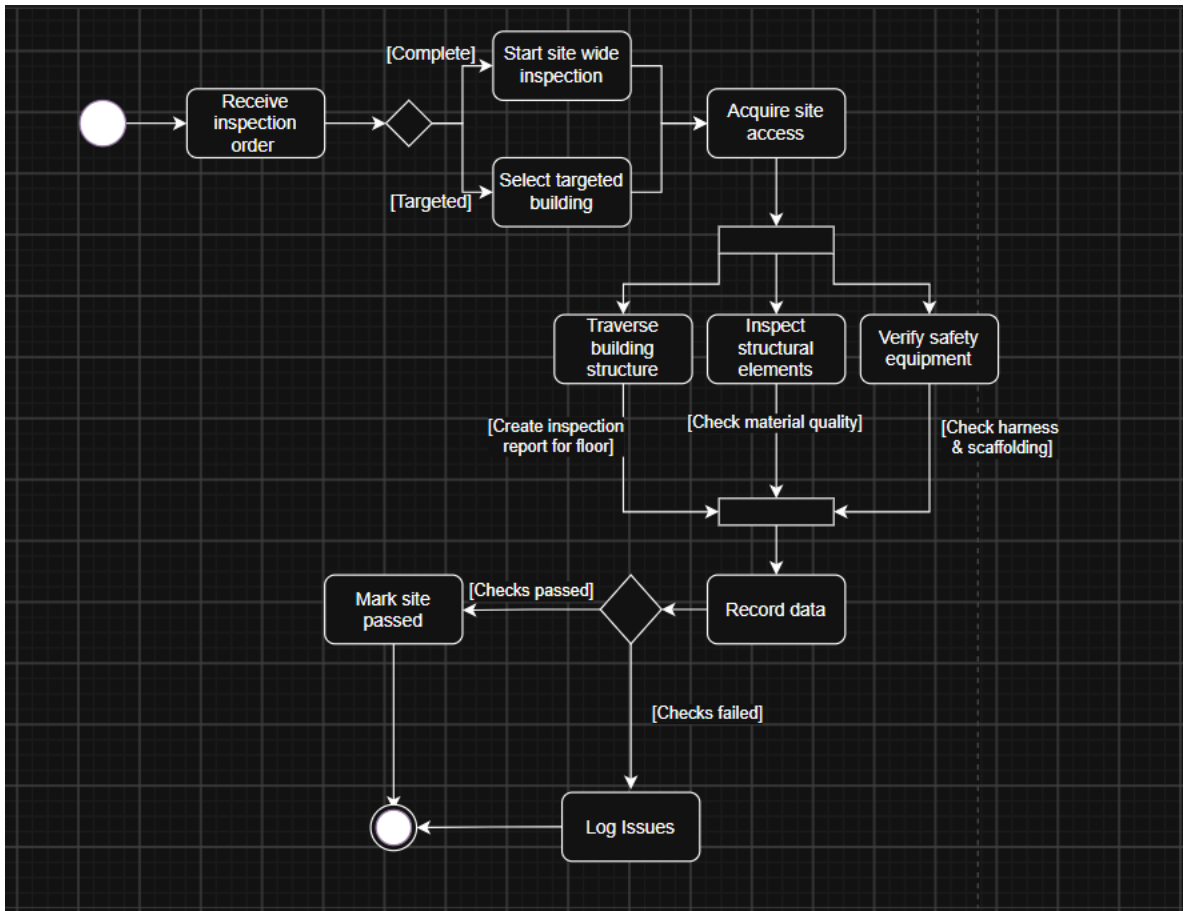
State diagram



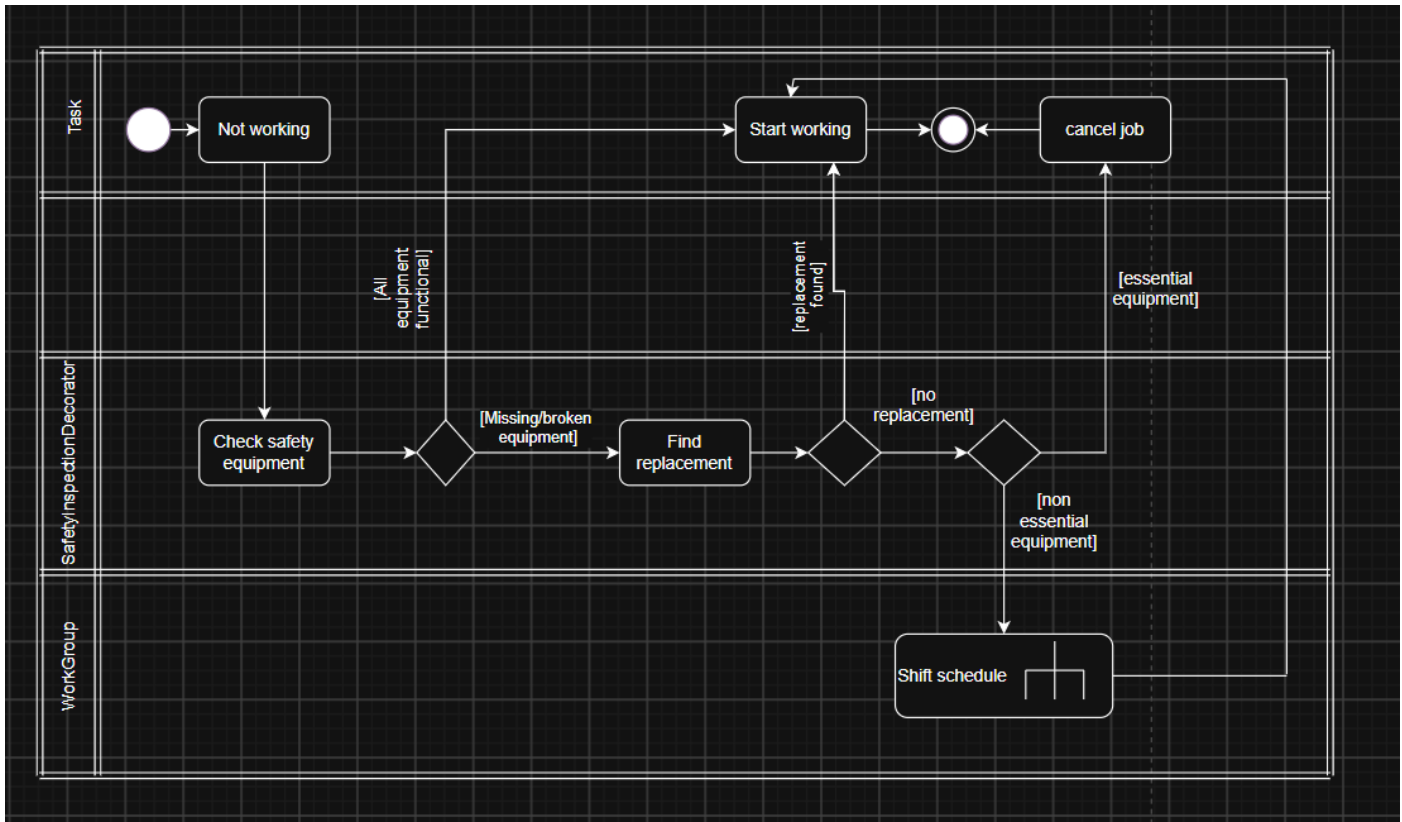
Activity diagram 1



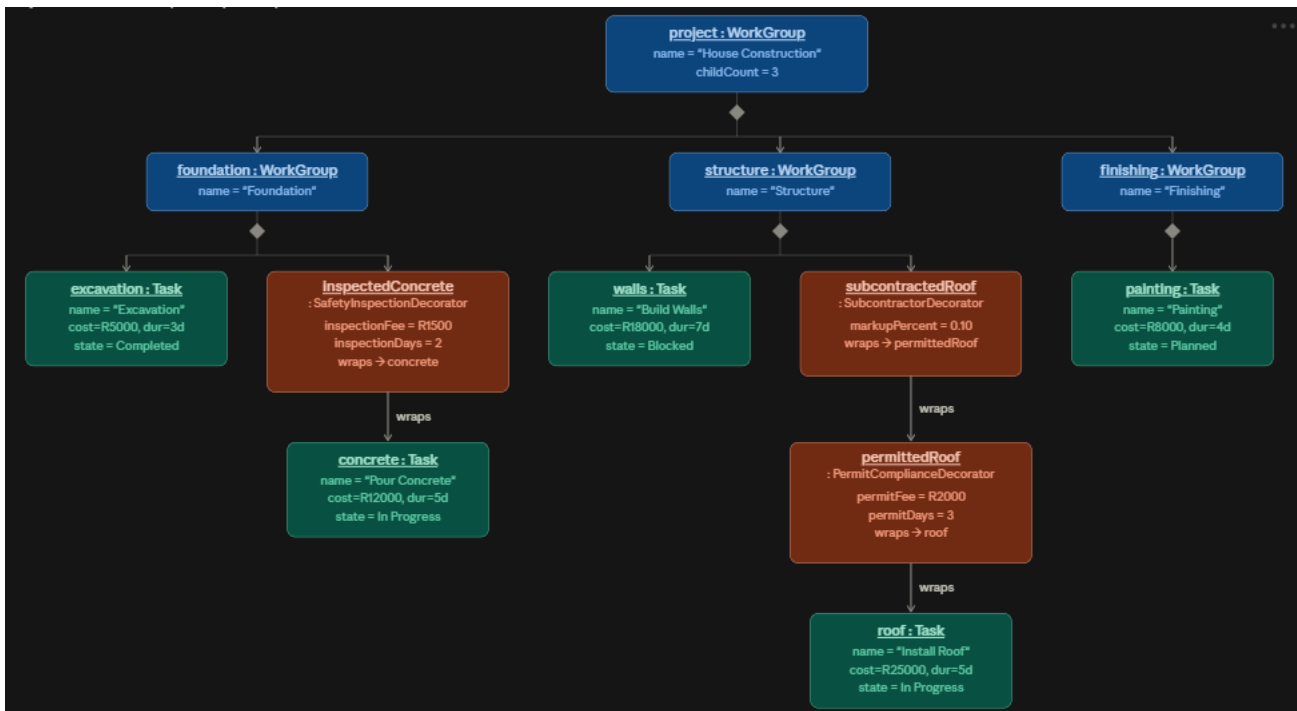
Activity diagram 2



Activity diagram 3



Object State diagram



Task 5

A bug we encountered during the implementation of this practical was that the Pending Inspection Iterator was not printing any tasks, even though they existed, like Pour Concrete and Install Roof were In Progress.

This was happening because the old iterator that we had tried something like:

```
Task* task = dynamic_cast<Task*>(child);
```

This attempted to convert each `WorkComponent*` directly into a `Task*`. However, Pour Concrete and Install Roof were not stored directly in their `WorkGroups` as `Task*` objects, but they were wrapped by decorators. Therefore, when the iterator encountered one of these objects, `child` pointed to a decorator rather than a `Task`, and since a decorator is not a `Task`, `dynamic_cast<Task*>(child);` returned `nullptr`.

```
./taskforge
Task blocked: Waiting for materials

=== CONSTRUCTION PROJECT ===
House Construction
Foundation
  Excavation State: Completed Cost: R5000 Duration: 3 days
  Pour Concrete State: In Progress Cost: R12000 Duration: 5 days
  - Safety Inspection: $1500 for 2 days
Structure
  Build Walls State: Blocked Cost: R18000 Duration: 7 days
  Install Roof State: In Progress Cost: R25000 Duration: 5 days
  - Permit Compliance: $2000 for 3 days
  - Subcontractor: 10% markup
Finishing
  Painting State: Planned Cost: R8000 Duration: 4 days

Total project cost: 74200
Project duration: 8 days

=== DEPTH FIRST TRAVERSAL ===
Foundation
Excavation
Pour Concrete
Structure
Build Walls
Install Roof
Finishing
Painting

=== PENDING INSPECTION ===

=== STATE CHANGES ===
Walls: Blocked
Walls after resume: In Progress
Walls after complete: Completed

=== PENDING INSPECTION AFTER STATE CHANGES ===
```

```
15         Task* task = dynamic_cast<Task*>(child);
(gdb) print child->getName()
$3 = "Pour Concrete"
(gdb) next
17         if (task != nullptr && task->getStateName() == "In Progress")
(gdb) print task
$4 = (Task *) 0x0
(gdb)
```

So to correct this problem, we introduced the polymorphic operation: `needsInspection()`, that determines whether a task is In Progress or not.

```
bool Task::needsInspection() const {  
    return getStateName() == "In Progress";  
}
```

Now decorators can forward `needsInspection()` to the `WorkComponent` they wrap, and iterator can therefore simply call:

```
if (child->needsInspection()) {  
    snapshot.push_back(child);  
}
```

Thus this allows the inspection check to travel through any decorators until it reaches the underlying Task. After this correction, Pour Concrete and Install Roof appeared correctly in the pending inspection traversal.

Valgrind:

```
-leak-kinds=all ./taskforge  
=== PENDING INSPECTION AFTER STATE CHANGES ===  
Pour Concrete  
Install Roof  
==74004==  
==74004== HEAP SUMMARY:  
==74004==    in use at exit: 0 bytes in 0 blocks  
==74004==   total heap usage: 37 allocs, 37 frees, 75,991 bytes allocated  
==74004==  
==74004== All heap blocks were freed -- no leaks are possible  
==74004==  
==74004== For lists of detected and suppressed errors, rerun with: -s  
==74004== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```